

邏輯系統

HW1

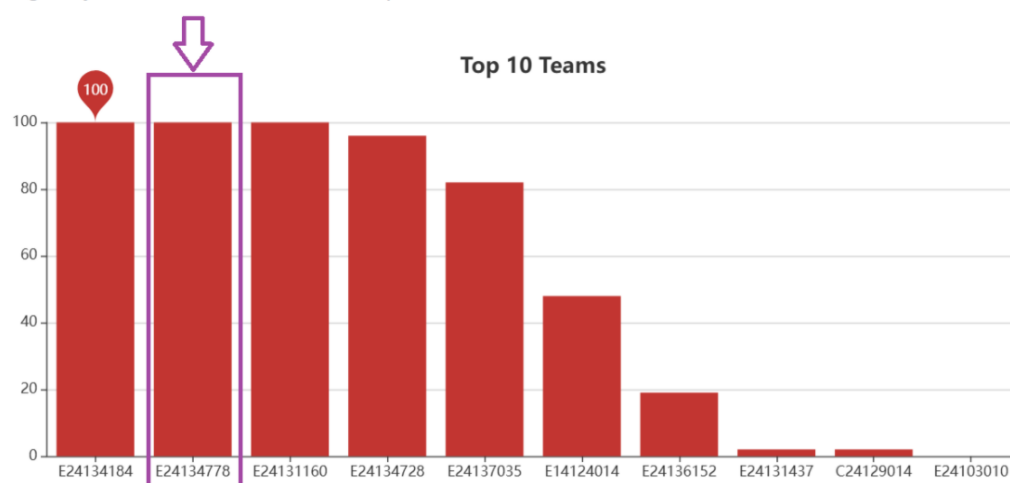
```
numVar = 3
minterms = 1 7
dontcares = 3 6

numVar = 3
kmap = [
  [ '0', '0', 'x', '0' ],
  [ '1', 'x', '1', '0' ]
]
primes = ["a'c", "ab", "bc"]
essentials = ["a'c"]
minimumSop = ["a'c", "ab"]

dontcares = 3 10 15

numVar = 4
kmap = [
  [ '0', '0', '0', '0' ],
  [ '0', '1', '1', '0' ],
  [ 'x', '1', 'x', '1' ],
  [ '1', '0', '1', 'x' ]
]
primes = ["ac", "b'c", "bd", "cd"]
essentials = ["ac", "b'c", "bd"]
minimumSop = ["bd", "ac", "b'c"]
```

2025 Logic System HW1 - An Auto K-map Solver



姓名：彭靖凱

學號：E24134778

授課教授：陳中和 教授

一、作業目標

本次作業的目的是

1. 讓學生實作教科書上所教的 K-map 自動化簡化過程。
2. 請撰寫一個程式，能夠處理 2 至 4 個變數的 K-map。
3. 程式需找出 Prime Implicants、Essential Prime Implicants 並顯示最小 SOP (Sum of Product) 表示式。

二、程式說明

1. 解析輸入參數 (numVar、minterms、dontcares)：

(1) 數解析 numVar (int)：

代表 K-map 的變數數量。有效範圍是 2, 3, 4。根據變數數量會決定 K-map 的大小與排列方式：

numVar=2 變數陣列：2×2

numVar=3 變數陣列：2×4

numVar=4 變數陣列：4×4

(2) minterms (vector<int>)：

表示輸入的標記為 1 的索引每個整數代表一組變數組合。

(3) minterms (vector<int>) 表示輸入的標記為 x 的索引每個整數代表一組變數組合。

例如

numVar = 4

minterms = [1, 3, 5, 7, 9] (如圖一標記為 1)

dontcares = [6, 12, 13] (如圖一標記為 x)

程式碼功能對照說明：

main() 主函式：

(1) cin >> numVar;

→ 讀取變數數量 (2~4)

(2) getline() 讀取

minterms 與 dontcares

→ 使用字串串流分別儲存至 vector<int>

(3) solveKmap()

→ 呼叫主程式處理函式，進行 K-map 計算

輸出：kmap 表格 (顯示每格為 '1', '0', 'x')

2. 建立 K-map 表格

(1) 根據變數數決定 K-map 表格大小

(2) 使用 Gray Code (格雷碼) 對應 K-map 行列

(3) 逐一將 minterm、dontcares 與其他狀況填入格子

Gray Code：

00 → 01 → 11 → 10 每次只變動 一個位元

(4) 建立 K-map → kmap[][] = ('0'/'1'/'x')

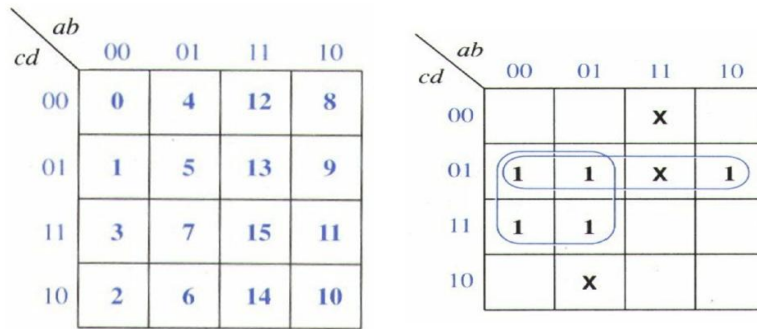
如右圖這個步驟是把輸入的 minterms 和 dontcares 對應到一個二維的 K-map 陣列 kmap[r][c] 中。每個位置會根據輸入被標記為：

'1'：代表 minterm，必須覆蓋

'x'：代表 don't care，可選擇是否合併

'0'：其他沒指定的輸入 (不是 minterm 也不是 dontcares)

```
int rows = (numVar == 2) ? 2 : (numVar == 3) ? 2 : 4;
int cols = (numVar == 2) ? 2 : (numVar == 3) ? 4 : 4;
// Initialize kmap 2D array with '0'
sol->kmap = new char* [rows];
for (int i = 0; i < rows; ++i) {
    sol->kmap[i] = new char[cols];
    fill(sol->kmap[i], sol->kmap[i] + cols, '0');
}
vector<char> kmapVals(1 << numVar, '0');
for (int d : dontcares) kmapVals[d] = 'x';
for (int m : minterms) kmapVals[m] = '1';
// Fill K-map table using Gray code position mapping
for (int i = 0; i < (1 << numVar); ++i) {
    auto [r, c] = getKmapPos(i, numVar);
    if (r < rows && c < cols) {
        sol->kmap[r][c] = kmapVals[i];
    }
}
```



圖一、陣列座標與 Minterms and dontcares 標記示意圖

(5)K-map Gray Code 對應表

下列依序為 numVar = 2, numVar = 3 與 numVar = 4 K-map 對應表

numVar = 2 (a, b)

b ↓ / a →	0	1
0	0 (00)	2 (10)
1	1 (01)	3 (11)

numVar = 3 (a, b, c)

c ↓ / ab →	00	01	11	10
0	0 (000)	2 (010)	6 (110)	4 (100)
1	1 (001)	3 (011)	7 (111)	5 (101)

numVar = 4 (a, b, c, d)

cd ↓ / ab →	00	01	11	10
00	0 (0000)	4 (0100)	12 (1100)	8 (1000)
01	1 (0001)	5 (0101)	13 (1101)	9 (1001)
11	3 (0011)	7 (0111)	15 (1111)	11 (1011)

範例：

以下為 numVar = 3 時，minterms = {1, 2, 4, 6}，dontcares = {0, 7} 的 K-map 表格範

例：(十進位位置)內容(0:不是 minterm 也不是 dontcare 1:minterm, x: dontcare)

c ↓ / ab →	00	01	11	10
0	(0)x(dontcare)	(2)1(minterm)	(6)1(minterm)	(4)1
1	(1)1(minterm)	(3)0	(7)x(dontcare)	(5)0

3. 找出 Prime Implicants (P.I.)

函式：findPrimeImplicants(...)

程式重點段落（如右圖）：

位元合併方式（K-map 法則合併）主要應用

用於 Karnaugh Map（卡諾圖）圖形上的相鄰格子合併，依照圖上「相鄰的 1 或 X」，進行群組合併。

合併邏輯：

- (1) 每次合併的格子數必須是 2, 4, 8... (2 的次方)
- (2) 只合併相鄰且只有一個位元不同的 minterm。
- (3) 這種方式是純圖形判斷，而不是比對編號位元。

程式流程說明：

- (1) 合併 minterms 與 dontcare
- (2) 所有 '1' 和 'x' 對應成 binary 字串（例如 000, 001）
- (3) 嘗試合併只相差一位元的兩個字串
- (4) 若合併成功，該位置設為 '-'，代表該變數可忽略（"000" 與 "001" → "00-")
- (5) 重複此過程直到不能再合併為止
- (6) 合併後的結果即為 Prime Implicant（主項）

```
for (size_t i = 0; i < keys.size(); ++i) {
    for (size_t j = i + 1; j < keys.size(); ++j) {
        string merged = mergeBinary(keys[i], keys[j]);
        if (!merged.empty()) {
            // Merge two groups if they differ by one bit
            nextGroups[merged] = groups[keys[i]];
            nextGroups[merged].insert(nextGroups[merged].end(),
                                      groups[keys[j]].begin(), groups[keys[j]].end());
            marked.insert(keys[i]);
            marked.insert(keys[j]);
            thisRound.insert(merged);
            done = false;
        }
    }
}
```

4. 找出 Essential Prime Implicants（必要主項）

函式：findEssentials(...)

功能說明：

- (1) 針對每一個 minterm，列出能覆蓋它的所有 implicants
- (2) 若某 Prime Implicants 中 implicant 只被一個覆蓋一次，該項為 Essential Prime Implicant
- (3) 最終轉為 SOP 格式並輸出

註：

轉換成 SOP 字串："0-1" 這類 binary+dash 字串，轉換成 SOP (Sum of Products) 格式如 "a'c"

函式名稱：string binToTerm(const string& bin)

5. 求出最小 SOP 表示式 (Minimum SOP)

函式：findMinimumCover(...)

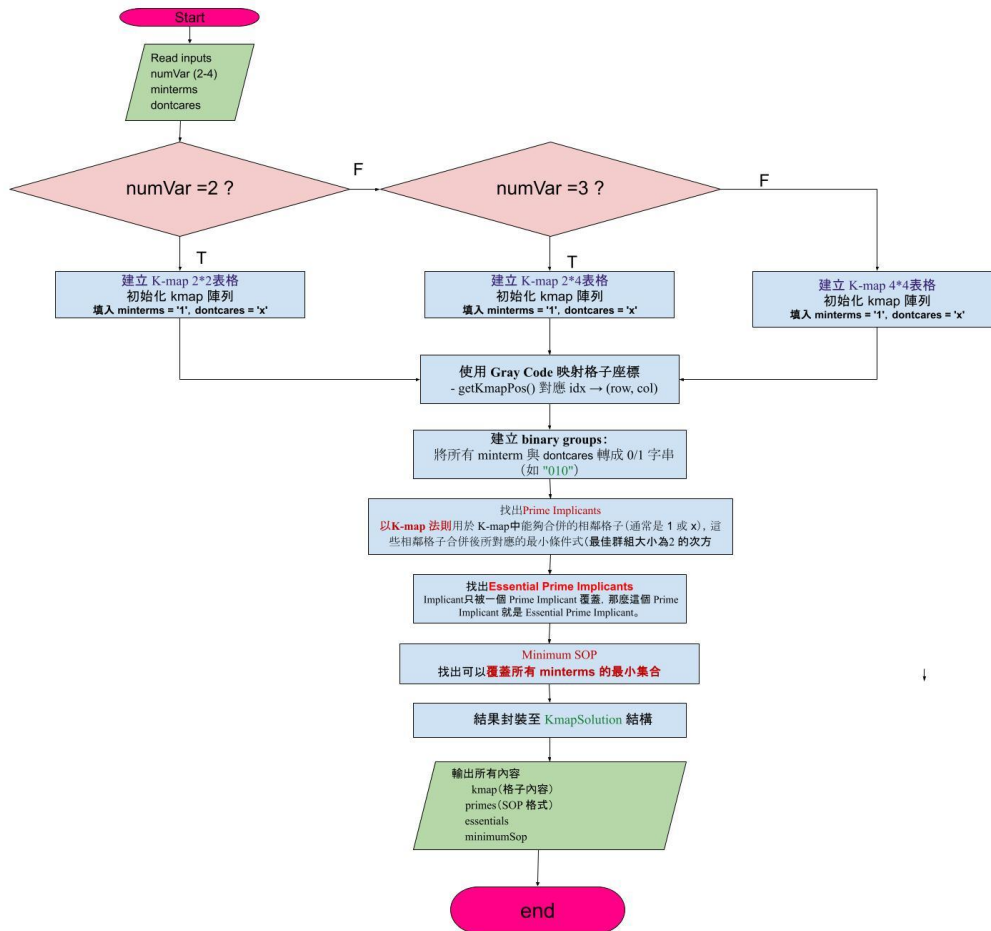
功能說明：

- (1) 使用 bitmask 檢查所有 prime implicants 的子集合，找出可以**覆蓋所有**

minterms 的最小集合

- (2) 將該集合轉為 SOP 字串（例如 "a'c"）

三、程式流程圖



上圖呈現了 K-map 求解流程圖，顯示了根據不同變數數量 (numVar) 進行 K-map 求解的步驟。下面是這張流程圖的文字描述：流程描述：

1. 開始 (Start)：

- 開始程序，讀取輸入：

- (1) numVar (變數數量，可以是 2、3 或 4)。
- (2) minterms (minterms 的索引)。
- (3) dontcares (dontcares 的索引)。

2. 分支條件：

- 根據 numVar 的值，決定如何處理：

- (1) 如果 numVar = 2，則選擇處理 2-variable K-map，並使用對應的 K-map 格式 2*2。
- (2) 如果 numVar = 3，則選擇處理 3-variable K-map，並使用對應的 K-map 格式 2*4。
- (3) 如果 numVar = 4，則選擇處理 4-variable K-map，並使用對應的 K-map 格式 4*4。

3. 填充 K-map :

- 將 minterms 和 dontcares 填充到 K-map 中，這會在 K-map 中設置 1（對應於 minterms）和 x（對應於 dontcares）。

4. Gray Code 計算 :

- 使用 Gray Code 計算格子的行和列。
 - (1) 利用 getKmapPos() 函數來確定每個格子的行 (row) 和列 (col) 位置。

5. 找出 Binary Groups :

- 將 K-map 中的相鄰格子（即相鄰的 1 或 x）進行合併，這些格子將組成新的 **binary groups**。

6. 找出 Prime Implicants (PI) :

- 根據組合後的格子，找出 K-map 中的 **Prime Implicants**，這些是合併的最小條件式。

7. 找出 Essential Prime Implicants (EPI) :

- 如果某個 Implicant 只被一個 Prime Implicant 覆蓋，那麼這個 Prime Implicant 就是 Essential Prime Implicant。

8. minimumSop :

- 根據 **Essential Prime Implicants** 和其他 **Prime Implicants**，進行最小化 SOP（和積型）化簡，得到最簡的邏輯表達式。
minimumSop 為可以覆蓋所有 minterms 的最小集合。

9. 輸出結果 :

- 輸出 KmapSolution 結果，其中包含：
 - (1) kmap：填充過的 K-map。
 - (2) primes：所有 Prime Implicants。
 - (3) essentials：所有 Essential Prime Implicants。
 - (4) minimumSop：最小化的 SOP 表達式。

10. 結束 (End) :

- 程式執行完成，返回結果。

四、資料結構說明

1. KmapSolution 結構體 (Struct KmapSolution)

成員變數名稱	資料型別	作用說明
numVar	int	儲存變數數量 (2~4)
kmap	char**	儲存 K-map 表格 (以字元陣列表示)
primes	vector<string>	儲存所有 prime implicants
essentials	vector<string>	儲存 essential prime implicants
minimumSop	vector<string>	儲存最小 SOP 解

2. 主程式變數說明 (main.cpp)

變數名稱	資料型別	作用說明	所在位置
numVar	int	使用者輸入的變數數量 (2~4)	main()
minterms	vector<int>	儲存輸入的 minterm 索引值	main()
dontcares	vector<int>	儲存輸入的 don't care 索引值	main()
sol	KmapSolution*	儲存求解後的 K-map 結果與資訊	main()

3. 核心函式說明

函式名稱	參數	回傳型別	作用說明
solveKmap	(int, vector<int>, vector<int>)	KmapSolution*	主解題函式，傳回 K-map 解與簡化結果
toBinary	(int, int)	string	將整數轉為固定長度的二進位字串
mergeBinary	(string, string)	string	合併兩個只差一位的二進位字串

binToTerm	(string)	string	將二進位轉為 SOP 表示法 (如：a'b)
getKmapPos	(int, int)	pair<int, int>	將 index 轉為對 應在 K-map 中的 (row, col)
findPrimeImplicants	(int, vector<int>, vector<int>)	vector<string>	找出所有 prime implicants
findEssentials	(vector<string>, vector<int>, int)	vector<string>	找出 essential prime implicants
findMinimumCover	(vector<string>, vector<int>, int)	vector<string>	找出能覆蓋所有 minterm 的最小 SOP 表示

五、編譯與執行結果(3 個 Case)

1. 編譯程式

編譯環境:作業系統：Windows 10 / 11

- (1) 開發工具：Microsoft Visual Studio 2022
- (2) C++ 標準：ISO C++ 20 (/std:c++20)
- (3) 編譯器：MSVC (Microsoft Visual C++)



執行流程

- 程式會要求使用者輸入：
 - 變數數量 (numVar)
 - minterms 清單
 - don't cares 清單
- 系統會自動建立並輸出：
 - K-map 表格
 - Prime implicants
 - Essential prime implicants
 - Minimum SOP 結果

執行結果說明

Case1:

```
Microsoft Visual Studio 1 x +  
  
numVar = 2  
minterms = 0 3  
dontcares = 1  
  
numVar = 2  
kmap = [  
  [ '1', '0' ],  
  [ 'x', '1' ]  
]  
primes = ["a", "b"]  
essentials = ["a", "b"]  
minimumSop = ["a", "b"]
```

執行分析：

1. **numVar = 2 :**

K-map 有兩個變數，a 和 b，因此它是一個 **2x2 的 K-map**。

2. **minterms = 0, 3 :**

K-map 中 **0** 和 **3** 的格子是 **minterms**，因此這些位置會被填上 **1**。

3. **dontcares = 1 :**

這表示 **dontcare** 位置是 **1**，因此該位置可以任意視為 **1** 或 **0**。這樣的填充通常是用 **x** 表示。

4.K-map 填充：

根據 **numVar = 2**，你有一個 **2x2 的 K-map**：

(1) **minterms** 填充：對應於 **minterms = {0, 3}**，K-map 在位置 **0** 和 **3** 被設為 **1**。

(2) **dontcares** 填充：對應於 **dontcares = {1}**，K-map 在位置 **1** 被設為 **x**。

5.Primes (Prime Implicants) :

右圖 Prime Implicants 是指 K-map 中能夠合併的相鄰格子 (通常是 **1** 或 **x**)，這些相鄰格子合併後所對應的最小條件式 (最佳的群組大小是 **2 的次方**)。這個範例中，兩個 Prime Implicants 是：

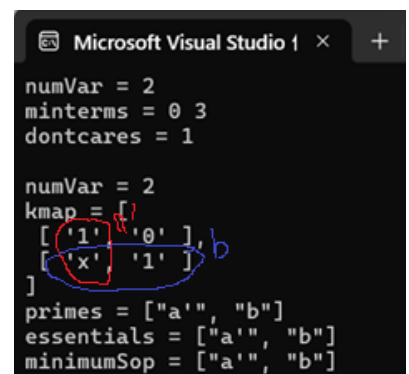
(1)a'：這個條件式覆蓋了 **minterms {0}** 的所有相鄰格子

(a=0,b=0)與(a=0,b=1) $\Rightarrow$ a0 $\Rightarrow$ a'

(2)b：這個條件式覆蓋了 **minterms {3}** 的所有相鄰格子

((a=0,b=1)與(a=1,b=1) $\Rightarrow$ b1 $\Rightarrow$ b

所以 **Primes**=["a'", "b"]



```
numVar = 2
minterms = 0 3
dontcares = 1

numVar = 2
kmap = [
  ['1', '0'],
  ['x', '1']
]
primes = ["a'", "b"]
essentials = ["a'", "b"]
minimumSop = ["a'", "b"]
```

6.Essentials (Essential Prime Implicants) :

Essential Prime Implicants 是最小化邏輯式時，必須保留的 Prime Implicants。這個範例中，**由於每個格子都有唯一的 Prime Implicant**，因此：
Essentials = ["a'", "b"]

7.Minimum SOP :

SOP (Sum of Products) 是將所有 Essential Prime Implicants 進行和積化簡。

a'+ b 可以找出 **覆蓋所有 minterms{0, 3} 的最小集合**

Minimum SOP = ["a'", "b"]

Case2:

```
Microsoft Visual Studio 1 × + v
numVar = 3
minterms = 1 7
dontcares = 3 6

numVar = 3
kmap = [
  [ '0', '0', 'x', '0' ],
  [ '1', 'x', '1', '0' ]
]
primes = ["a'c", "ab", "bc"]
essentials = ["a'c"]
minimumSop = ["a'c", "ab"]
```

執行分析：

1. **numVar = 3 :**

這表示 K-map 有三個變數 ab 和 c，因此我們將使用一個 2x4 的 K-map，總共會有 8 個格子，對應於所有 3 變數的組合。

2. **minterms = 1, 7 :**

這代表 minterms = 1 和 7 的格子會被設為 1。這些位置對應於 K-map 中的 1。

3. **dontcares = 3, 6 :**

這代表 dontcare = 3 和 6 的格子會被設為 x，這些位置可以任意視為 1 或 0。

4.K-map 填充：

根據 numVar = 3，我們需要使用 3 變數的 K-map，因此 K-map 的大小會是 2x4 的結構。根據 minterms 和 dontcares 來填充這個 K-map。

(1) minterms 填充：對應於 minterms = {1, 7}，K-map 在位置 1 和 7 被設為 1。

(2) dontcares 填充：對應於 dontcares = {3, 6}，K-map 在位置 3 和 6 被設為 x。

5.Prime Implicants (PI) :

右圖 Prime Implicants 是 K-map 中相鄰的格子合併後的最小條件。這個範例中，3 個 Prime Implicants 是：

(1) a'c：這個條件式覆蓋了 minterms {1}，對應於

ab=00, c=1 和 ab=01, c=1。

=>a0c1=> a'c

(2) ab：這個條件式覆蓋了 minterms {7}，對應於

ab=11, c=0 和 ab=11, c=1

=>a1b1=> ab

(3) bc：這個條件式覆蓋了 minterms {7}，對應於

ab=01, c=1 和 ab=11, c=1

=>b1c1=> bc

```
numVar = 3
minterms = 1 7
dontcares = 3 6

numVar = 3
kmap = [
  [ '0', '0', 'x', '0' ],
  [ '1', 'x', '1', '0' ]
]
primes = ["a'c", "ab", "bc"]
essentials = ["a'c"]
minimumSop = ["a'c", "ab"]
```

所以 **Primes**=["a'c","ab","bc"]

6.Essentials (Essential Prime Implicants) :

a'c : minterms {1}是唯覆蓋 1 次的條件。

Essentials = ["a'c"]

7.Minimum SOP :

a'c+ ab or a'c+ bc 可以找出 **覆蓋所有 minterms{1,7} 的最小集合**

Minimum SOP = ["a'c ", "ab"] or Minimum SOP = ["a'c ", "bc"]

Case3:

```
Microsoft Visual Studio 1 x + -
numVar = 4
minterms = 2 5 7 11 13 14
dontcares = 3 10 15

numVar = 4
kmap = [
  [ '0', '0', '0', '0' ],
  [ '0', '1', '1', '0' ],
  [ 'x', '1', 'x', '1' ],
  [ '1', '0', '1', 'x' ]
]
primes = ["ac", "b'c", "bd", "cd"]
essentials = ["ac", "b'c", "bd"]
minimumSop = ["bd", "ac", "b'c"]
```

執行分析：

1.numVar = 4 :

這表示 K-map 有 4 個變數 ab 和 cd，因此需要使用一個 4x4 的 K-map。這會對應於所有 4 變數的組合，總共有 16 個格子。

2.minterms = {2, 5, 7, 11, 13, 14} :

這代表 minterms 在位置 2, 5, 7, 11, 13, 14 會被設為 1。

3.dontcares = {3, 10, 15} :

這代表 dontcare 位置是 3, 10, 15，這些格子會被設為 x。這些格子可以視為 1 或 0，用來進行簡化。

4.K-map 填充：

根據 numVar = 4，我們需要使用 4 變數的 K-map，因此 K-map 的大小會是 4x4。根據 minterms 和 dontcares 來填充這個 K-map。

minterms 填充：對應於 minterms = {2, 5, 7, 11, 13, 14}，K-map 在這些位置被設為 1。

dontcares 填充：對應於 dontcares = {3, 10, 15}，K-map 在這些位置被設為 x。

5.Prime Implicants (PI) :

Prime Implicants 是 K-map 中相鄰的格子合併後的最小條件。這個範例中，找到了 4 個 Prime Implicants：

(1) ac：這個條件式覆蓋了 minterms {11, 14}，對應於 K-map 中的

ab=11, ab=10 和 **cd=11, cd=10**。

$\Rightarrow a1c1 \rightarrow ac$

- (2) $b'c$ ：這個條件式覆蓋了 minterms $\{2, 11\}$ ，對應於 K-map 中

$ab=00, ab=10$ 和 $cd=11, cd=10$ 。

$\Rightarrow b0c1 \rightarrow b'c$

- (3) bd ：這個條件式覆蓋了 minterms $\{5, 7, 13\}$ ，對應於

$ab=01, ab=11$ 和 $cd=01, cd=11$ 。

$\Rightarrow b1d1 \rightarrow bd$

- (4) cd ：這個條件式覆蓋了 minterms $\{7, 11\}$ ，對應於

$ab=00, 01, 11, 10$ 和 $cd=11$

$\Rightarrow c1d1 \rightarrow cd$

所以 $\text{primes} = ["ac", "b'c", "bd", "cd"]$

6. Essentials (Essential Prime Implicants)：

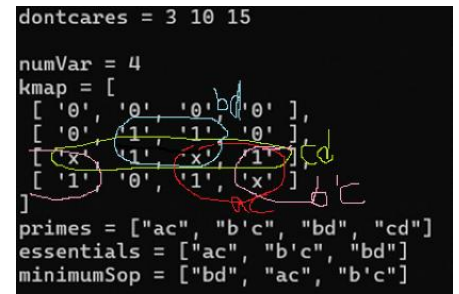
參考 Prime Implicants (PI) 發現 minterms $\{2, 5, 13, 14\}$ 是唯覆蓋 1 次的條件。

所以 $\text{Essentials} = ["ac", "bd", "b'c"]$

7. Minimum SOP：

$ac + bd + b'c$ 可以找出 覆蓋所有 minterms $\{2, 5, 7, 11, 13, 14\}$ 的最小集合

Minimum SOP = $["ac", "bd", "b'c"]$

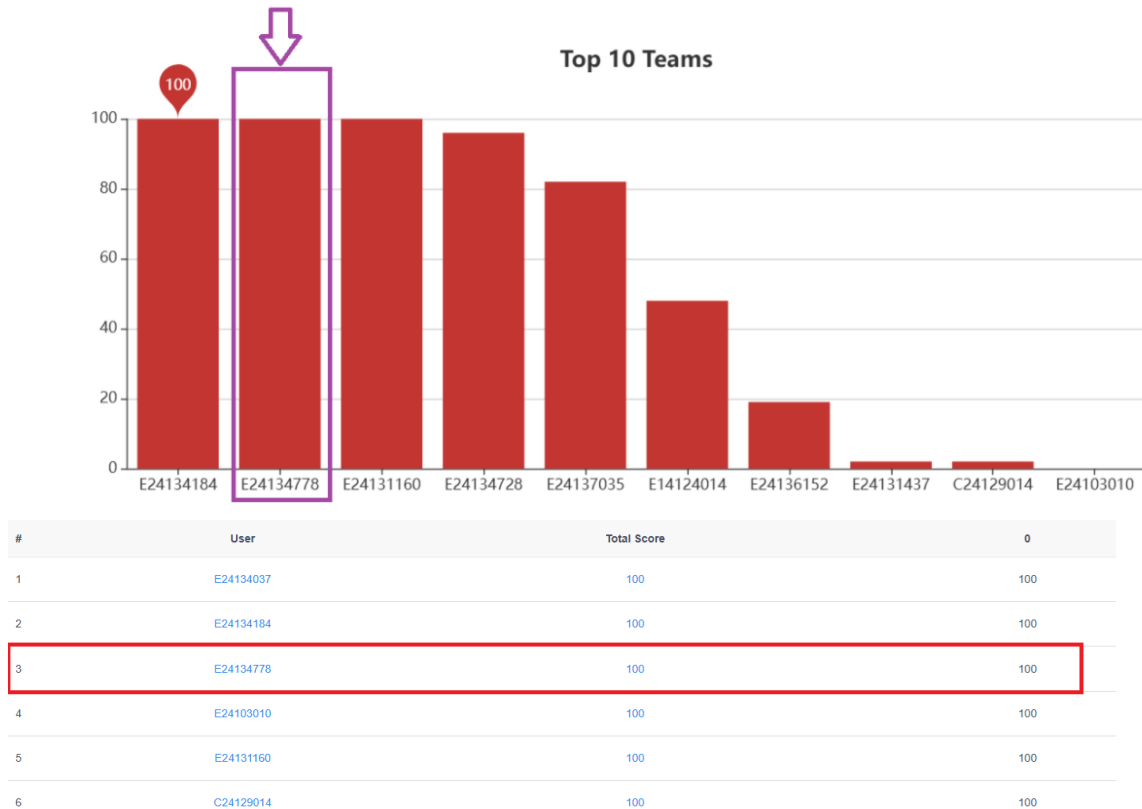


六、將程式碼上傳至 CASOJ 系統進行測試

(<https://casoj.ee.ncku.edu.tw/contest>)

測試結果如下圖 (100分)

2025 Logic System HW1 - An Auto K-map Solver



七、結論

程式成功實現了 K-map 的自動化簡化過程，能夠處理 2 到 4 變數的 K-map，並根據使用者輸入的 minterms 和 dontcares 計算 Prime Implicants、Essential Prime Implicants 以及最小化的 SOP 表達式。

這次作業讓我對 **K-map** 的簡化過程有了更深的理解，特別是在自動化簡化邏輯式、處理多變數情況下的技巧。這不僅幫助我熟悉了程式設計和數位邏輯，更讓我在面對邏輯簡化的問題時，能夠有效應用數學原則和程式邏輯。